Agentic Tree Search

A Database-Native Approach to Agentic Knowledge Retrieval

Technical White Paper

April 2026

Table of Contents

Right-click and select 'Update Field' to populate the table of contents.

1. Executive Summary

Enterprise AI assistants increasingly rely on autonomous retrieval from internal knowledge bases, and the prevailing pattern — Agentic File Search — treats the knowledge store as a tree of Markdown files navigated by an agent through filesystem tools. While adequate for prototypes, this pattern imposes a silent tax on production deployments: filesystem coupling, opaque governance, poor scalability, and fragile integration with enterprise stacks. This white paper introduces Agentic Tree Search, a database-native pattern that preserves the ergonomic simplicity of a tree-based mental model while relocating the entire knowledge graph into a relational schema. The approach is grounded on two tables (KnowledgeNode with self-join, plus an optional KnowledgeChunk) and three KernelFunction tools (GetKnowledgeMap, ReadNode, SearchNodes). A hybrid ReadNode tool transparently serves either the full content of an atomic node or a chunk index for long documents, keeping the agent interface stable and minimal. Agentic Tree Search delivers centralized governance, referential integrity, native full-text and vector search extensibility, multi-tenant isolation, versioning, and seamless integration with existing SQL-based enterprise platforms — without expanding the agent's tool surface or retraining prompts.

2. The Problem: Limits of Agentic File Search

Agentic File Search treats the host filesystem as the system of record, and this choice propagates constraints throughout the stack. Every reasonable operation — adding a policy, fixing a typo, promoting an approved article — requires filesystem access on the application host, a redeploy, or a pipeline execution. In modern containerized or PaaS environments, writable persistent filesystems are either unavailable or discouraged, and ephemeral file layers invalidate the premise of a stable knowledge source.

Governance is equally brittle. When the knowledge base lives as flat files, there is no application-level control surface that records who modified what and when; access is mediated by operating-system ACLs that rarely match the business identity model. Git partially fills the gap but introduces a parallel permission system that business users cannot realistically operate, and pull-request review does not map naturally to an editorial approval workflow.

Search is the third weak point. Filesystem-based agents rely on grep-style substring search, which cannot leverage stemming, thesauri, language analysis, or vector similarity without introducing an external indexing pipeline that shadows the file tree. As the corpus grows, list_directory calls become heavy, duplicate content across trees is impossible to detect, and the agent's map of the domain degrades into a brittle set of hardcoded paths. Finally, the pattern scales poorly beyond a single team: multi-tenancy forces parallel folder hierarchies and duplicated ACLs, and there is no natural way to attach metadata — version, approver, embedding vector — to a node without inventing side-car files.

3. The Solution: Agentic Tree Search

Agentic Tree Search preserves the tree metaphor that agents reason about well, but relocates the tree into a relational schema with a self-referencing foreign key. A single KnowledgeNode table

models both folders and files through a NodeType column; ParentId realizes the hierarchy; Content and Tags carry the payload for leaf nodes. A recursive Common Table Expression reconstructs a precomputed FullPath during retrieval, giving the agent immediate structural orientation without walking the tree node by node.

```
CREATE TABLE KnowledgeNode (
    Id        UNIQUEIDENTIFIER PRIMARY KEY DEFAULT NEWID(),
    ParentId  UNIQUEIDENTIFIER NULL REFERENCES KnowledgeNode(Id),
    Name      NVARCHAR(200)  NOT NULL,
    NodeType  NVARCHAR(10)   NOT NULL,
    Content   NVARCHAR(MAX)  NULL,
    Tags      NVARCHAR(500)  NULL,
    UpdatedAt DATETIME2      NOT NULL DEFAULT GETDATE()
);
```

Long articles that would exceed a practical context window are optionally split into a secondary KnowledgeChunk table, indexed by a ChunkIndex and annotated with the original Markdown heading. Structure (KnowledgeNode) and fragmented content (KnowledgeChunk) are thus cleanly separated, and the presence of chunks is decided per node rather than enforced corpus-wide. Node names are chosen semantically — "Billing / Invoicing / VAT on Gas" rather than opaque filenames — so the knowledge map itself behaves as an implicit prompt, guiding the agent toward the right subtree before any content is read. A depth of three levels (Domain → Macro-topic → Topic) has proved sufficient in practice and keeps the flat map human-readable.

The agent interacts with this structure through exactly three tools, exposed as Microsoft Semantic Kernel functions. GetKnowledgeMap returns the full flattened tree, including IDs and paths, in a single call. ReadNode is a hybrid: invoked on an atomic node it returns the content directly, while on a chunked node it returns the index of sections and expects a follow-up call with a chunkIndex argument. SearchNodes performs a transverse lookup on Content and Tags. The hybrid behaviour of ReadNode is intentionally invisible to the agent, which uses a stable signature regardless of whether a node is atomic or fragmented — the branching lives inside the tool, not inside the prompt.

4. Advantages over Agentic File Search

Moving the knowledge tree into the database is not a mere refactoring: it changes what is structurally possible. The following advantages are cumulative, and each one is hard or impossible to replicate on a plain filesystem without re-implementing a fragment of a database engine.

4.1 Centralized and Application-Level Management

With Agentic Tree Search, the knowledge base is mutated through ordinary SQL statements — INSERT, UPDATE, DELETE — executed by any application with a connection string. There is no filesystem access, no redeploy, no CI/CD pipeline required to promote a change. A back-office web UI, an ETL job, or a content-moderation microservice can all coexist as first-class editors. The practical effect is that editorial cadence decouples from release cadence, and a typo fix is a one-line SQL update rather than a pull request against an infrastructure repository.

4.2 Structural Integrity by Schema

Referential integrity guarantees that no child node can exist without its parent, and the NodeType column provides a typed vocabulary enforced at the database layer. An UpdatedAt trigger, a CHECK constraint on NodeType, and a unique index on (ParentId, Name) jointly eliminate entire classes of defects — orphans, duplicates, malformed folder/file combinations — that on a filesystem can only be detected by a periodic consistency job. The schema is, in effect, executable documentation of what a well-formed knowledge tree is.

4.3 Natively Extensible Search

The default SearchNodes implementation uses a LIKE clause across Content and Tags. Because the search surface is a single SQL statement wrapped in a tool, it is trivially upgradable: replacing LIKE with CONTAINS or FREETEXT activates SQL Server Full-Text Search — with language analyzers, stemming, and thesauri — without altering the agent's tool signature or prompt. No external index, no Lucene sidecar, no file watcher is required. The agent keeps calling SearchNodes; the quality of results improves underneath.

4.4 Transparent, Opt-In Chunking

Chunking is an engineering concern, not a property of the knowledge domain, and Agentic Tree Search treats it accordingly. A node becomes chunked only when its content length justifies it; other nodes remain atomic. The agent never inspects this distinction — ReadNode returns either a full document or an index of sections, and the agent reacts to the shape of the response. On a physical filesystem, chunking would require either splitting a document into several files (polluting the tree) or introducing an additional tool that reveals the chunking layer to the agent.

4.5 Efficient Agent Orientation

GetKnowledgeMap returns the complete flattened tree in a single call, with IDs and precomputed paths ready to be consumed by subsequent ReadNode and SearchNodes invocations. The equivalent reconstruction on a filesystem requires N list_directory calls — one per folder — and each call returns only a partial view. The database pattern thus compresses a depth-bounded recursive traversal into a single query, which is both faster and cheaper in terms of tokens consumed by the agent.

4.6 Native Versioning and Audit

Because nodes are rows, auditing is a column away. Adding Version, CreatedBy, ApprovedBy, ApprovedAt, or RetiredAt columns extends the governance model without introducing new infrastructure, and temporal tables or Change Data Capture provide automatic history. Reproducing the same fidelity on a filesystem requires Git — with its separate access model — or a bespoke history store. The relational approach turns compliance from a project into a schema migration.

4.7 Multi-Tenancy and Application-Level Permissions

A single OrganizationId or TeamId column on KnowledgeNode isolates knowledge per tenant while reusing the application's existing identity and authorization infrastructure. Row-level security policies enforce the isolation at the database layer, so a shared SearchNodes tool naturally returns only the rows the current principal is allowed to see. On a filesystem, the same outcome would require

parallel directory trees, mirrored ACLs at the operating-system level, and a runtime that can impersonate the calling user — a substantially heavier apparatus for an equivalent result.

4.8 Seamless Integration with the Existing Stack

In most enterprise environments — including multi-utility enterprise contexts where SQL Server is already a first-class citizen — Agentic Tree Search adds exactly zero new infrastructure components. The knowledge base lives next to the domain data, shares the backup and disaster-recovery strategy, and inherits observability tooling. Agentic File Search, by contrast, presumes writable filesystem access on the application host, which is frequently unavailable on container platforms, serverless hosts, or managed PaaS runtimes. The database-native pattern removes an entire class of deployment conversations.

4.9 Linear Path to Semantic RAG

Leaf nodes and chunks are the natural unit of embedding. Adding an EmbeddingVector column — or mirroring the chunks into a vector store such as Qdrant — is an additive evolution that preserves the tree structure and the three-tool interface. Agentic Tree Search thus becomes the intermediate layer between a curated, human-readable knowledge base and a vector-based RAG subsystem. Hybrid retrieval strategies — structural first, semantic second — emerge naturally, with the tree providing precision and the embeddings providing recall.

4.10 Minimal Agent Interface

The entire interaction surface is three tools. The agent does not need to know about physical paths, file extensions, directory separators, or chunk boundaries, and prompts remain stable even if the underlying schema, storage engine, or chunking strategy evolves. A new backing store can be introduced behind the KernelFunction boundary without retraining or prompt engineering. In production terms, this is the property that makes the pattern durable: the contract between agent and knowledge is narrow, explicit, and versionable.

5. Direct Comparison

The following table summarises the two approaches along the dimensions that most influence enterprise adoption decisions.

Dimension	Agentic File Search	Agentic Tree Search
Required infrastructure	Writable filesystem on the app host	Existing relational database
Content updates	File edits + redeploy or CI/CD pipeline	SQL INSERT/UPDATE/DELETE at runtime
Search	grep-style substring match	LIKE today, Full-Text / vector tomorrow
Chunking	Requires extra files or an extra tool	Opt-in per node, transparent to agent
Versioning	External (Git) or custom	Native columns / temporal tables / CDC
Permissions	OS-level ACLs	Application identity + row-level security
Multi-tenancy	Parallel folder trees, mirrored	Single column (OrganizationId)

	ACLs	
Enterprise integration	Difficult on containers / PaaS	Seamless where SQL is already deployed
Path to RAG	Requires parallel indexing pipeline	Additive (EmbeddingVector / vector store)
Agent tool surface	N filesystem tools	3 stable KernelFunctions
Operational complexity	Grows with corpus size	Grows with schema richness, not size

6. When Agentic File Search Is Still the Right Choice

Intellectual honesty requires acknowledging that the database-native approach is not universally superior. Agentic File Search remains the pragmatic choice when a Markdown repository on Git is already the organisation's system of record for knowledge — typically a developer-facing documentation site where pull-request review is the approved editorial workflow. It also fits teams that rely on desktop Markdown editors, local previewing, and conventional branch-based collaboration: introducing a database would break a well-working authoring loop. Environments that lack a relational database entirely — edge deployments, strictly static sites, early-stage prototypes — gain little from relocating the knowledge into infrastructure that must first be provisioned. Finally, single-user proofs of concept built to validate the agentic loop itself benefit from the zero-setup nature of the filesystem. The decision, in short, is governed by the lifecycle stage and the surrounding stack rather than by raw capability.

7. Reference Architecture

The reference architecture centers on the two-table schema introduced in Section 3. KnowledgeNode owns the hierarchy and the atomic payload; KnowledgeChunk owns the fragmented payload for long nodes, with a cascading foreign key to guarantee that orphan chunks cannot survive their parent. A recursive CTE computes FullPath on demand, which keeps the schema normalised while giving the agent a precomputed representation at retrieval time.

```
WITH CTE AS (
  SELECT Id, CAST(Name AS NVARCHAR(MAX)) AS FullPath
  FROM KnowledgeNode WHERE ParentId IS NULL
  UNION ALL
  SELECT k.Id, c.FullPath + N'/' + k.Name
  FROM KnowledgeNode k JOIN CTE c ON k.ParentId = c.Id
)
SELECT Id, NodeType, FullPath FROM CTE ORDER BY FullPath;
```

On the application side, three KernelFunctions expose the behaviour. GetKnowledgeMap executes the recursive CTE and returns an ordered, indented representation of the tree. ReadNode dispatches internally between atomic read, chunk-index read, and chunk-content read; the branching is implementation-private.

```
[KernelFunction]
public async Task<string> ReadNode(Guid nodeId, int? chunkIndex = null) {
```

```
    if (chunkIndex.HasValue) return await ReadChunkAsync(nodeId, chunkIndex.Value);
    var count = await GetChunkCountAsync(nodeId);
    return count > 0
        ? await GetNodeIndexAsync(nodeId, count)
        : await ReadNodeContentAsync(nodeId);
}
```

SearchNodes performs a parameterised LIKE across Content and Tags, returning a match excerpt alongside each hit so the agent can select targets without a second ReadNode round-trip when the excerpt is sufficient. A companion component, the KnowledgeChunkingService, is responsible for producing KnowledgeChunk rows from long Content values — typically by splitting on Markdown headings and annotating each chunk with heading and token count. The service is optional: a knowledge base composed entirely of short atomic nodes requires no chunking infrastructure.

Three canonical agent flows emerge. In the atomic case, the agent invokes GetKnowledgeMap, locates the relevant node, calls ReadNode(id), and responds. In the chunked case, the agent calls ReadNode(id), receives the section index, and issues a follow-up ReadNode(id, chunkIndex) to retrieve the precise fragment. In the search case, the agent calls SearchNodes(query), inspects excerpts, and then uses ReadNode on the chosen result. The reference implementation uses C# on .NET, Microsoft Semantic Kernel as the orchestration layer, SQL Server as the durable store, and a locally hosted language model such as Qwen3-8B served through Ollama — but none of these choices are load-bearing: the pattern is deliberately portable across SDKs and engines.

8. Conclusions

Agentic Tree Search formalises a migration that several enterprise teams are already performing ad hoc: from filesystem-as-knowledge-base toward database-as-knowledge-base, while retaining the tree abstraction that agents navigate naturally. By reducing the agent-facing surface to three tools and absorbing the complexity — hierarchy, chunking, search — into a small relational schema, the pattern aligns knowledge management with the governance, observability, and integration standards that the rest of the enterprise stack already meets. It is a pragmatic pattern for production, not a theoretical reframing.

The forward trajectory is equally straightforward. Embedding vectors attach to leaf nodes or chunks to enable hybrid structural-plus-semantic retrieval. Editorial workflow columns — draft, reviewed, approved, published — turn the knowledge base into an auditable editorial system without adding new services. Multilingual variants emerge as a Language column plus a TranslationOf reference, and tenant isolation composes with row-level security already used elsewhere. In each direction the evolution is additive: the three-tool contract between agent and knowledge remains stable, and what changes lives behind the boundary where it belongs. Agentic Tree Search is thus positioned to become the default pattern for knowledge retrieval in agentic enterprise systems — a database-native replacement for a filesystem-native habit.